

Another DataGridView Printer

Introduction

Microsoft created a tremendously useful control with the DataGridView, but left out a particularly important detail: Printing the DataGridView. There were several projects available to print a DataGridView at the time I created this, but none did what I wanted. I wanted to not have objects, controls or code from the printer object sprinkled throughout my code and I needed to be able to print individual pages, current selection or everything in the DataGridView.

The end result of trying to solve this problem is this project, Another DataGridView Printer (DGVPrinter). The DGVPrinter object will print DataGridViews that are wider than a single page in a columnar fashion. To do this, DGVPrinter will print all the pages for the columns that fit onto the first page, then all the pages for the columns that overflow onto the next page, and so on. Currently, there is no limit on the width of the overall DataGridView or for individual columns, with the single provision that a column wider than one page will be automatically resized to print on a single page. DGVPrinter will also handle cells that are deeper than a single page. DGVPrinter will print the entire contents of the cell, even if it spans multiple pages, and DGVPrinter will handle rows that span multiple pages both width wise and depth wise at the same time. When a row spans multiple pages, you can set a list of 'fixed' columns that will appear on every page for that row. This can help correlate the same row across multiple pages. DGVPrinter will also handle DataGridViews with the RightToLeft setting turned on.

The DGVPrinter object provides several properties to control how the DataGridView columns are treated on the printed page. The default is to take the column widths from the source DataGridView. Column widths can be overridden during the printing process to provide finer control of how your DataGridView fits on the printed page. The DGVPrinter object also supports "proportional" column widths, where the printed portion of the source DataGridView is spread to cover the entire page. When the proportional column widths setting is turned on and individual column widths are also specified, the overridden column widths are respected. The result is that you can have a mix of fixed and proportional column widths as you deem necessary to look good on the printed page.

DGVPrinter will also print images from image columns. The display of the image respects the ImageLayout column property to clip, stretch or scale the image to fit within the displayed cell in the printout. The image print also respects the Alignment cell style for placing the image in the output cell. In order to scale the rows properly for image printing, the "RowHeightSetting" property should be set to "RowHeightSetting.CellHeight". Checkbox columns now print with a graphic checkbox that is checked or not based on the column value.

Finally, the DGVPrinter printing engine can be called directly with a Graphics object to print within a Graphics context that you control (i.e. within another printing process).

Using the DataGridViewPrinter

The DGVPrinter has three modes of use. The first two involve the DGVPrinter object controlling the print process; the third is for printing a data grid view within an external print process. The first two modes of using the DGVPrinter are very similar and involve printing as a single method call or printing as a two separate calls, one for the print dialog and one for the actual print processing. The third mode exists to allow the use of DGVPrinter's logic within your own graphical or printing process.

Include the DGVPrinter in Your Project

There are two ways to include the DGVPrinter in your project. The first is to put the provided .DLL file into your project's "bin" or other directory and then provide a reference to it. This should work for any language. The second method is to simply include the DGVPrinter.cs file in your project. While simpler, the second method only works for C# projects.

Setting up to use the DGVPrinter

The DGVPrinter object can be used directly with a DataGridView, and does not require any scaffolding or extraneous code to be added to your project. The only bit of code that you need (other than the actual calls to use the object) is a "using" or "imports" statement to include the namespace for the class.

```
//  
// The using block statement.  
//  
Using DGVPrinterHelper;
```

Figure 1. Using Statement – C#

```
//  
// The Imports block statement.  
//  
Imports DGVPrinterHelper
```

Figure 2. Imports Statement - VB

Simple use of the DGVPrinter

To use the DGVPrinter object, you simply create it when you need it, set its properties and call the print routine. The DGVPrinter supports direct printing or printing through a print preview dialog. For example, you might want to handle a “Print” button on your window’s toolbar:

```
//
// Printing the DataGridView Control
// in response to a toolbar button press
//
private void printToolStripButton_Click(object sender, EventArgs e)
{
    DGVPrinter printer = new DGVPrinter();
    printer.Title = "DataGridView Report";
    printer.SubTitle = "An Easy to Use DataGridView Printing Object";
    printer.SubTitleFormatFlags = StringFormatFlags.LineLimit |
        StringFormatFlags.NoClip;

    printer.PageNumbers = true;
    printer.PageNumberInHeader = false;
    printer.ColumnWidth = DGVPrinter.ColumnWidthSetting.Porportional;
    printer.HeaderCellAlignment = StringAlignment.Near;
    printer.Footer = "Your Company Name Here";
    printer.FooterSpacing = 15;

    printer.PrintDataGridView(datagridviewController);
}
```

Figure 3. Simple Printing Example – C#

```
'
' Printing the DataGridView Control
' in response to a toolbar button press
'
Private Sub btnPrintGridview_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles btnPrintGridView.Click
    Dim Printer = New DGVPrinter
    Printer.Title = "DataGridView Report"
    Printer.SubTitle = "An Easy to Use DataGridView Printing Object"
    Printer.SubTitleFormatFlags = StringFormatFlags.LineLimit Or StringFormatFlags.NoClip
    Printer.PageNumbers = True
    Printer.PageNumberInHeader = False
    Printer.ColumnWidth = DGVPrinter.ColumnWidthSetting.Porportional
    Printer.HeaderCellAlignment = StringAlignment.Near
    Printer.Footer = "Your Company Name Here"
    Printer.FooterSpacing = 15
    Printer.PrintDataGridView(Me.DataGridView1)
End Sub
```

Figure 4. Simple Printing Example – VB

This is a fairly simple example of setting the header and footer text and printing the DataGridView without a preview process.

Complex use of the DGVPrinter

The DGVPrinter also supports separation of the basic two step printing process – displaying the print dialog and performing the print page process. This is to increase your options for integrating DGVPrinter into your program; saving and re-applying printer settings allows you the option to provide your users with a better experience by keeping their printing preferences.

```
//  
// Printing the DataGridView Control  
// in response to a toolbar button press – the myprintsettings and mypagesettings objects are objects used by the local  
// program to save printer and page settings  
//  
private void printToolStripButton_Click(object sender, EventArgs e)  
{  
    DGVPrinter printer = new DGVPrinter();  
    printer.Title = "DataGridView Report";  
    printer.SubTitle = "An Easy to Use DataGridView Printing Object";  
    printer.SubTitleFormatFlags = StringFormatFlags.LineLimit |  
        StringFormatFlags.NoClip;  
    printer.PageNumbers = true;  
    printer.PageNumberInHeader = false;  
    printer.ColumnWidth = DGVPrinter.ColumnWidthSetting.Porportional;  
    printer.HeaderCellAlignment = StringAlignment.Near;  
    printer.Footer = "Your Company Name Here";  
    printer.FooterSpacing = 15;  
  
    // use saved settings  
    if (null != myprintsettings)  
        printer.PrintDocument.PrinterSettings = myprintsettings;  
    if (null != mypagesettings)  
        printer.PrintDocument.DefaultPageSettings = mypagesettings;  
  
    if (DialogResult.OK == printer.DisplayPrintDialog()) // you may replace  
        DisplayPrintDialog() with your own print dialog  
    {  
        // save users' settings  
        myprintsettings = printer.PrinterSettings;  
        mypagesettings = printer.PageSettings;  
  
        // print without redisplaying the printdialog  
        printer.PrintNoDisplay(datagridviewController);  
    }  
}
```

Figure 5. Advanced Printing Example

It's also possible to implement your own print dialog. This can help your program to provide a consistent look and feel to your users.

Embedded Printing with the DataGridViewPrinter

DGVPrinter provides two options for Embedded Printing, which is printing to a graphics context provided by the calling program. The original interface was designed for printing grids within a single region provided in the calling parameters. The new interface allows for greater control by providing access to DGVPrinter's Begin Print and Print Page event handlers. These events can be generated by the calling program to print grids that span multiple pages to graphics contexts provided by the calling program.

Embedded Printing for Small Grids

The DGVPrinter originally provided an interface for printing a grid to a graphics context within the bounds of a provided area.

```
//  
// Printing the DataGridView Control  
// printing within a separate printing process  
//  
  
// Print to a graphics context provided in PrintPageEventArgs  
private void printDataGridView(object sender, PrintPageEventArgs e)  
{  
    print = new DGVPrinter();  
    print.RowHeight = DGVPrinter.RowHeightSetting.CellHeight;  
    e.HasMorePages = print.EmbeddedPrint(RequestList, e.Graphics,  
        new Rectangle(180, 480, 400, 120));  
}
```

Figure 6. Embedded Printing Example – creating and subscribing to the Begin Print and Print Page events

In this example, the graphics context is provided by the PrintPageEventArgs, however you can generate your own by creating graphics context from a BitMap object. This works well for small datagridview objects or when the entire grid can be printed to a single 'page'.

Embedded Printing Process for Multipage Printing

Establishing Printing Events and Handlers for Multipage Printing

For more complex applications, where the DataGridView needs to be printed over multiple pages, the Begin Print and Print Page events handlers can be called by raising your own events in the calling program.

```
//  
// Printing the DataGridView Control  
// establish events for begin print and print page  
//  
public delegate void myprintpageeventhandler(Object sender, PrintPageEventArgs e);  
public event myprintpageeventhandler myprintpage;  
protected virtual void Onmyprintpageevent(PrintPageEventArgs e)  
{  
    if (!(null == myprintpage))  
    {  
        myprintpage(this, e);  
    }  
}  
  
public delegate void mybeginprinteventhandler(Object sender, PrintEventArgs e);  
public event mybeginprinteventhandler mybeginprint;  
protected virtual void Onmybeginprintevent(PrintEventArgs e)  
{  
    if (!(null == mybeginprint))  
    {  
        mybeginprint(this, e);  
    }  
}  
  
// create printer object  
var print = new DGVPrinter();  
  
// subscribe begin print to my begin print event  
mybeginprint += new mybeginprinteventhandler(print.BeginPrintEventHandler);  
  
// subscribe print page to my print page event  
myprintpage += new myprintpageeventhandler(print.PrintPageEventHandler);
```

Figure 7. Embedded Printing Example – creating and subscribing to the Begin Print and Print Page events

The first step in this process is to create and manage your own ‘Begin Print’ and ‘Print Page’ events. The figure above shows sample event management objects for both events. When the DGVPrinter object is created during your print process, the “BeginPrintEventHandler” and “PrintPageEventHandler” event handlers in DGVPrinter need to be subscribed to their respective events.

Create Graphics Contexts and Printing via Raising Events

The next step in the Embedded printing process is to set the page and margin sizes as well as create a graphics context within which DGVPainter will draw pages. The “EmbeddedPrintMultipageSetup” routine must be called to setup DGVPainter for the Embedded print process before any events are raised. The Begin Print event is raised once at the beginning of printing. The Print Page event is raised once per page, and results in DGVPainter printing the next page to the graphics context passed in the PrintPageEventArgs object. You can save multiple images, as in the example, or you can use the resulting page

```
//
// Printing the DataGridView Control
// print the grid view to my graphics context - a series of bitmap images
//

bool cont = true;

// page size and margins setup
Rectangle pagesize = new Rectangle(0, 0, 800, 300);
Rectangle pagemargins = new Rectangle(0, 0, 0, 0);

// create graphical destination for the page
var pageimage = new Bitmap(pagesize.Width, pagesize.Height);
Graphics page = Graphics.FromImage(pageimage);

// setup embedded printing
print.EmbeddedPrintMultipageSetup(RequestList, new Rectangle(pagemargins.X, pagemargins.Y,
    pagesize.Width, pagesize.Height));

// create print page event arguments
var evtargs = new PrintPageEventArgs(page, pagemargins, pagesize, new PageSettings());

// create array to hold printed page images
List<Bitmap> pages = new List<Bitmap>();

// raise begin printing event
Onmybeginprintevent(new PrintEventArgs());

while (cont)
{
    // raise print page event
    Onmyprintpageevent(evtargs);

    // keep 'has more pages' flag from print page processing
    cont = evtargs.HasMorePages;

    // save this image and get a new image
    pages.Add(pageimage);
    pageimage = new Bitmap(pagesize.Width, pagesize.Height);
    page = Graphics.FromImage(pageimage);

    // new event args to hold new graphics context
    evtargs = new PrintPageEventArgs(page, pagemargins, pagesize, new PageSettings());
}
```

Figure 8. Embedded Printing Example – Raising the Begin Print and Print Page events, and save each output page to a separate image

image immediately, then calling the Clear method on the graphics object and re-use the same graphics context and PrintPageEventArgs object. If you choose the latter method, the resulting image should be saved or used immediately to prevent being overwritten by the next Print Page event. You have access to all the existing controls available in DGVPainter.

DGVPrinter Document Properties

PrinterSettings PrintSettings

Provides access to the PrinterSettings class in the underlying PrintDocument. If you are overriding the display of the print dialog, this object must be correctly set up by the calling process. For advanced control of the printing process.

PrintDialogSettingsClass PrintDialogSettings

Provides access to the print dialog settings. For advanced control of the printing process.

- **bool AllowSelection:** When true, the “Selection” option is displayed on the print dialog Page Range. The default is ‘true’.
- **bool AllowSomePages:** When true, the “Pages” option is displayed on the print dialog Page Range. The default is ‘true’.
- **bool AllowCurrentPage:** When true, the “Current Page” option is displayed on the print dialog Page Range. The default is ‘true’. This option does not apply if the UseEXDialog option is false.
- **bool AllowPrintToFile:** When true, the “Print to file” option is displayed on the print dialog. The default is ‘false’.
- **bool ShowHelp:** When true, the “Help” option is displayed on the print dialog. The default is ‘true’. This option only applies to versions prior to Windows 2000.
- **bool ShowNetwork:** When true, the “Network” option is displayed on the print dialog. The default is ‘true’. This option only applies to versions prior to Windows 2000.
- **bool UseEXDialog:** When true, the print dialog shown is the style for Windows XP and later. This option should be ‘true’ for proper functioning in Windows Vista and later versions. The default is ‘true’.

String PrinterName

Allows the caller to set the printer name. Overrides the default printer name that is presented in the printer settings dialog

PrintDocument printDocument

Provides access to the underlying PrintDocument. If you are overriding the display of the print dialog, this object must be correctly set up by the calling process. For advanced control of the printing process.

Icon PreviewDialogIcon

Allow the calling process to set the icon displayed in the upper left corner of the print preview dialog.

Form Owner

Allow the calling process to set the “owner” of the print preview window.

Double PrintPreviewZoom

Provides the ability to override the default zoom level of the print preview display.

Boolean PrintHeader

Flag indicating whether or not print the Page Header, including Title, Subtitle and Page Number (if in the header).

Boolean PrintFooter

Flag indicating whether or not to print the Page Footer.

Boolean? PrintColumnHeaders

Flag indicating whether or not to print the column header line. Defaults to the visibility of column headers in the source DataGridView.

Boolean? PrintRowHeaders

Flag indicating whether or not to print the row header cells. Note that an empty cell value for the row header will cause the row headers to not print. Defaults to “False” instead of the “RowHeadersVisible” property of the source DataGridView in order to preserve current and expected functionality.

Boolean KeepRowsTogether

Flag indicating whether or not to start a new page for a row that would otherwise run off the bottom of a page. The default is “true”.

Float KeepRowsTogetherTolerance

When ‘KeepRowsTogether’ is set to false, partial rows smaller than this tolerance are printed on the next page. This can be useful to prevent tiny row fragments from appearing at the bottom of a page. The default value is 15.

DGVPrinter Event Handlers

void BeginPrintEventHandler

Handles the Begin Print event. The Begin Print event is sent automatically by Windows printing objects and print drivers. You will need to subscribe this to a Begin Print event when you are raising your own Begin Print event during Embedded printing processes.

void PrintPageEventHandler

Handles the Print Page event. The Print Page event is sent automatically by Windows printing objects and print drivers for each page printed. You will need to subscribe this to a Print Page event when you are raising your own Print Page events during Embedded printing processes. The PrintPageEventArgs.HasMorePages value is set to false when printing is completed.

DGVPrinter Title Properties

String Title

The text of the main title for the printout.

String DocName

The text of the document name. The document name is displayed in the printer queue and in print status dialogs.

Font TitleFont

The font to use to print the title text. The default is 18 point Tahoma.

Color TitleColor

Foreground color used to print the title text. The default is black.

StringFormat TitleFormat

Contains format settings such as alignment, trimming and the StringFormatFlags used to print the title text.

StringAlignment TitleAlignment

Allow the user to override the alignment of the title text. Default value is "Near".

StringFormatFlags TitleFormatFlags

Allow the user to override the format flags for printing the title. Default values are "NoWrap", "LineLimit", and "NoClip".

Float TitleSpacing

Number of pixels below the title to place the next element

Brush TitleBackground

Set a brush to use to fill the title background area. The title area is from margin to margin across the width of the page and the height of the drawn text.

Pen TitleBorder

Set a pen to use to draw a border around the title area. The title area is from margin to margin across the width of the page and the height of the drawn text.

PrintLocation TitlePrint

Set the pages on which to print the title. Available locations are FirstOnly, LastOnly, All and None.

DGVPrinter Subtitle Properties

String SubTitle

The text of the subtitle for the printout.

Font SubTitleFont

The font to use to print the subtitle text. The default is 12 point Tahoma.

Color SubTitleColor

Foreground color used to print the subtitle text. The default is black.

StringFormat SubTitleFormat

Contains format settings such as alignment, trimming and the StringFormatFlags used to print the subtitle text.

StringAlignment SubTitleAlignment

Allow the user to override the alignment of the subtitle text. Default value is "Near".

StringFormatFlags SubTitleFormatFlags

Allow the user to override the format flags for printing the subtitle. Default values are "NoWrap", "LineLimit", and "NoClip".

Float SubTitleSpacing

Number of pixels below the subtitle to place the next element.

Brush SubTitleBackground

Set a brush to use to fill the subtitle background area. The subtitle area is from margin to margin across the width of the page and the height of the drawn text.

Pen SubTitleBorder

Set a pen to use to draw a border around the subtitle area. The subtitle area is from margin to margin across the width of the page and the height of the drawn text.

PrintLocation SubTitlePrint

Set the pages on which to print the subtitle. Available locations are FirstOnly, LastOnly, All and None.

DGVPrinter Footer Properties

String Footer

The text of the footer for the printout. Default is no footer.

Font FooterFont

The font to use to print the footer text. The default is 10 point Tahoma.

Color FooterColor

Foreground color used to print the footer text. The default is black.

StringFormat FooterFormat

Contains format settings such as alignment, trimming and the StringFormatFlags used to print the footer text.

StringAlignment FooterAlignment

Allow the user to override the alignment of the footer text. Default value is "Center".

StringFormatFlags FooterFormatFlags

Allow the user to override the format flags for printing the footer. Default values are "NoWrap", "LineLimit", and "NoClip".

float FooterSpacing

Set the amount of whitespace above the footer text.

Brush FooterBackground

Set a brush to use to fill the footer background area. The footer area is from margin to margin across the width of the page and the height of the drawn text.

Pen FooterBorder

Set a pen to use to draw a border around the footer area. The footer area is from margin to margin across the width of the page and the height of the drawn text.

PrintLocation FooterPrint

Set the pages on which to print the footer. Available locations are FirstOnly, LastOnly, All and None.

DGVPrinter Page Numbering Properties**bool PageNumbers**

If true, page numbers will be included in the printout. Default is true.

Font PageNumberFont

The font to use to print the page numbers. The default is 8 point Tahoma.

Color PageNumberColor

Foreground color used to print the page numbers. The default is black.

StringFormat PageNumberFormat

Contains format settings such as alignment, trimming and the StringFormatFlags used to print the page numbers.

StringAlignment PageNumberAlignment

Allow the user to override the alignment of the page numbers. Default value is "Near".

StringFormatFlags PageNumberFormatFlags

Allow the user to override the format flags for printing the page numbers. Default values are "NoWrap", "LineLimit", and "NoClip".

bool PageNumberInHeader

If true, page numbers will be printed at the top of the page, otherwise page number will be printed at the bottom of the page. Default is "false".

bool PageNumberOnSeparateLine

If true, the page number will be printed on a separate line, if false, the page numbers will be printed on the same line as the header or footer. Default is "false".

bool ShowTotalPageNumber

If true the page number is printed as "N of M" where N is the current page number and M is the total page count. Default is "false".

String PageSeparator

The text separating the page number and the total page count when "ShowTotalPageNumber" is true. Default is English " of ".

String PageText

The text preceding the page number. Default is English "Page ".

String PartText

When printing a DataGridView that is wider than one page, the overflow pages are identified with the matching page number and a part number, i.e. "Page 2 – Part 1". This property holds the text used to separate the page numbers and part numbers. Default is " – Part ".

PrintLocation PageNumberPrint

Set the pages on which to print the page number. Available locations are FirstOnly, LastOnly, All and None.

DGVPrinter Header Cell Properties

Dictionary<string, DataGridViewCellStyle> ColumnHeaderStyles

Allows per column style overrides of the column headers. Defaults to the column header styles of the source DataGridView. Values are null until set (c.f. "ColumnStyles" below).

DataGridViewCellStyle RowHeaderCellStyle

Allows overriding the row header cell style. Defaults to the row header cell style in the source DataGridView. Value is null until set (c.f. "ColumnStyles" below).

StringFormat GetColumnHeaderCellFormat(DataGridView)

Extracts the column header cell format from the provided DataGridView as a default, and allows the caller to make any overrides for printing. Will be overridden by a DataGridViewCellStyle set through "ColumnHeaderStyles".

StringFormat GetRowHeaderCellFormat(DataGridView)

Extracts the row header cell format from the provided DataGridView as a default, and allows the caller to make any overrides for printing. Will be overridden by a DataGridViewCellStyle set through "RowHeaderStyles".

String RowHeaderCellDefaultText

Provides a default value to prevent the row header cell from being 'invisible' (i.e. 0 width) when no value is set in the row in the DataGridView. Defaults to one tab space.

StringAlignment HeaderCellAlignment

Allow the user to override the alignment of the column header cells. Deprecated, please use GetHeaderCellFormat().

StringFormatFlags HeaderCellFormatFlags

Allow the user to override the format flags for printing the column header cells. Deprecated, please use GetHeaderCellFormat().

DGVPrinter Cell Properties

StringFormat GetCellFormat(DataGridView)

Extracts the cell format from the provided DataGridView as a default, and allows the caller to make any overrides for printing. Will be overridden by a DataGridViewCellStyle set through "ColumnStyles".

StringAlignment CellAlignment

Allow the user to override the alignment of the column cells. Deprecated, please use GetCellFormat() or ColumnStyles.

StringFormatFlags CellFormatFlags

Allow the user to override the format flags for printing the column cells. Deprecated, please use GetCellFormat() or ColumnStyles.

Dictionary<String, float> ColumnWidths

Allow the printed width of a column to be overridden. To use, add the column name and the desired width to the dictionary list.

```
DGVPrinter printer = new DVGPrinter();
printer.ColumnWidths.Add("descriptionGridViewTextBox", 200);
```

Figure 9. Add a column width override

Dictionary<String, DataGridViewCellStyle> ColumnStyles

```
DGVPrinter printer = new DVGPrinter();
printer.ColumnStyles[RequestList.Columns[1].Name] = RequestList.DefaultCellStyle.Clone();
printer.ColumnStyles[RequestList.Columns[1].Name].Font = new Font("Arial", (float)12.5);
printer.ColumnStyles[RequestList.Columns[1].Name].Alignment =
    DataGridViewContentAlignment.MiddleCenter;
```

Figure 10. Add per-column style overrides

Allow per-column style overrides. To use, add the column name and the desired style for that column to the dictionary list. If “RequestList” is the name of our DataGridView object, this example shows setting the column style for the second column in the printout: Note the use of the “Clone()” method of the DataGridView’s DefaultCellStyle. If you do not clone the cell’s style object, you’ll be making changes to your DataGridView display as well as the printout.

Dictionary<String, DataGridViewCellStyle> AlternatingRowColumnStyles

Allow per-column style overrides for alternating row support. To use, add the column name and the desired style for that column to the dictionary list. This style will be used for odd-index numbered rows. If “RequestList” is the name of our DataGridView object, this example shows

```
// Copy the base style for our modified column style -
// c.f. the previous example, "Per-Column Style Overrides"
DataGridViewCellStyle Style = printer.ColumnStyles[RequestList.Columns[1].Name].Clone();

// Apply alternating row styling

// Apply the alternating row style from data grid
Style.ApplyStyle(RequestList.AlternatingRowsDefaultCellStyle);

// Apply our font changes
Style.Font = new Font("Arial", (float)12.5, FontStyle.Bold);

// Apply our alignment changes
Style.Alignment = DataGridViewContentAlignment.MiddleCenter;

// Set the alternating row style
printer.AlternatingRowColumnStyles[RequestList.Columns[1].Name] = Style;
```

Figure 11. Add per-column style overrides for Alternating Row styling

setting the alternating row column style for the second column in the printout: Note the use of the “Clone()” method to make a copy of the base column style. The “ApplyStyle” method adds the DataGridView’s existing Alternating Row style to our modified Column Style so that the DataGridView and the printout have similar Alternating Row styling. We can then further modify our Column Style as desired.

List<String> FixedColumns

List of column names that will be repeated on each page when a row spans multiple pages. These columns are printed on the left for left-to-right languages and on the right for right-to-left languages, based on the RightToLeft setting in the DataGridView. If an area of the grid is printed (i.e. the user chose to print selected columns or rows) that does not include the fixed columns, they will be added to the printout.

List<String> HideColumns

List of column names that will be excluded from the grid printout.

String BreakOnValueChange

Column name that is monitored for value changes. A page break is inserted when the value in this column changes. Sorting on this column is recommended. The default value is an empty String, which disables this function.

DGVPrinter Page Properties

Margins PrintMargins

The page margins used for the printout. Default is (60, 60, 40, 40) or (0,0,0,0) for Embedded printing.

PageSettings PageSettings

Allow the user access to the underlying PrintDocument’s PageSettings object. For advanced control of the printing process.

bool PorportionalColumns

If true, columns without a width override will be spread proportionally across the page to fill the page from margin to margin. Default is “false”. This property is deprecated. Please use the “ColumnWidth” property.

Alignment TableAlignment

Allow the printed DataGridView to be aligned to the left, right or center of the printed page. If PorportionalColumns is turned on, TableAlignment is ignored. Default is “NotSet”.

RowHeightSetting RowHeight

Determines the default height for a row. “RowHeightSetting.DataHeight” uses the calculated height of the text or image data, and generally results in a more compact print display.

“RowHeightSetting.CellHeight” uses the height of the cell as the default height. Note that this is independent of the ColumnWidth setting.

ColumnWidthSetting ColumnWidth

Determines the default width for columns in a row. “ColumnWidthSetting.DataWidth” uses the calculated width of the text or image data, and generally results in a more compact print display and uses less paper. “ColumnWidthSetting.CellWidth” uses the width of the cell as the default width. “ColumnWidthSetting.Porportional” will spread the columns proportionally across the page to fill the page from margin to margin. Note that this is independent of the RowHeight setting.

DGVPrinter Image Properties

DGVPrinter can display logos, watermarks or other images on the page. NOTE: All images are printed as “Watermarks”, that is, behind the text on the page.

IList<ImbeddedImage> ImbeddedImageList

The list of images that will be printed on the page. This is a list of “ImbeddedImage” structures that define where DGVPrinter should print the image on the page.

ImbeddedImage Class

Provides information on where DGVPrinter should display an image on the printed page.

```
DGVPrinter printer = new DGVPrinter();
DGVPrinter.ImbeddedImage iil = new DGVPrinter.ImbeddedImage();
iil.ImageAlignment = DGVPrinter.Alignment.NotSet;
iil.ImageLocation = DGVPrinter.Location.Absolute;
iil.ImageX = 0;
iil.ImageY = 0;
iil.theImage = new Bitmap("<full file name here>");
print.ImbeddedImageList.Add(iil);
```

Figure 12. Setting an image to display on the page

- **Image theImage:** The image that will be drawn on the page.
- **DGVPrinter.Location Location:** The location to print the image. Can be ‘Header’, ‘Footer’ or ‘Absolute’. ‘Header’ location will print at the top of the page (i.e. at the top margin), while ‘Footer’ will print the image at the bottom of the page (at the bottom margin). ‘Absolute’ allows you to position the image any place on the page. Note that ‘Absolute’ positioning does NOT account for the margins.
- **DGVPrinter.Alignment Alignment:** Can be set to ‘Left’, ‘Center’, ‘Right’ or ‘Not Set’. ‘Left’ and ‘Right’ position the image against the left and right margins, respectively. ‘Center’ centers the image. If you are using the ‘Absolute’ location, then the ‘Alignment’ value is ignored.
- **Int ImageX:** X value of the upper left hand corner of the image, used only for ‘Absolute’ image location.
- **Int ImageY:** Y value of the upper left hand corner of the image, used only for ‘Absolute’ image location.

DGVPrinter Methods

DGVPrinter()

Constructor for the object, takes no parameters.

void PrintDataGridView(DataGridView)

Simple interface; displays the print settings dialog to the user, and prints the provided DataGridView to the selected printer.

void PrintPreviewDataGridView(DataGridView)

Simple interface; displays the print settings dialog to the user, and then displays the resulting printout in the print preview dialog.

DialogResult DisplayPrintDialog()

Advanced interface; displays the print settings dialog to the user and returns the results to the caller.

void PrintNoDisplay(DataGridView)

Advanced interface; prints the provided DataGridView. The PrintSettings and PrintDocument must be ready and able to print, either through a previous call to the DisplayPrintDialog() method or similar process.

void PrintPreviewNoDisplay(DataGridView)

Advanced interface; displays the provided DataGridView in the print preview dialog. The PrintSettings and PrintDocument must be ready and able to print, either through a previous call to the DisplayPrintDialog() method or similar process.

bool EmbeddedPrint(DataGridView, Graphics, Rectangle)

Embedded print interface. DGVPrinter will print the provided DataGridView within the area of the provided Rectangle. Use this to print grids that will fit within the print area.

bool EmbeddedPrintMultipageSetup(DataGridView, Rectangle)

Embedded print interface. Sets up DGVPrinter to print within the area of the provided Rectangle. Use this to print multiple pages to your own graphics objects. Printing is accomplished by raising 'Begin Print' and 'Print Page' events after this set up routine has been called.

bool EnableLogging

Enables tracing within DGVPrinter. Logs are writing to a trace file that is in the current directory by default. Note that 'current' is in relation to the calling executable. The file name has the format 'DGVPrinter_yyyymmdd.Log'.

String LogDirectory

Changes the directory where the logging file will be written. The file name has the format 'DGVPrinter_yyyymmdd.Log'. Setting this value will enable logging if it is not already enabled.

Overriding Cell Printing

DGVPrinter will allow you to override the drawing of the cells being printed. To do this, you can subscribe to the OwnerDraw event. This event will be called for every cell, including column and row headers.

DGVCellDrawingEventArgs

This is the event arguments object that is passed to the OwnerDraw event delegate. It contains the following fields:

- **Graphics g:** the graphics object to use for all drawing. Note that this is the graphics context for the entire page, so be sure to use the correct coordinates when printing text or images.
- **RectangleF DrawingBounds:** the boundaries and location of the cell being drawn
- **DataGridViewCellStyle CellStyle:** the current cell drawing style
- **Int row:** the row index of the cell being drawn. A column header cell will have a value of -1
- **Int column:** the column index of the cell being drawn. A row header cell will have a value of -1
- **Boolean Handled:** set this to 'true' to indicate that drawing the cell is complete. Default value is 'false'. If 'false' is returned, DGVPrinter will print the cell.

Owner drawing the cell gives the user complete control over the look of the cell; however, the user is also responsible for drawing all aspects of the cell, including background and gridlines. The example below is an owner drawing procedure that rotates the column headers 180 degrees, and moves the printed header back into the bounds of the cell rectangle. Setting the RowHeight property to "CellHeight" and the "ColumnWidth" property to "CellWidth" is recommended when overriding cell drawing.

```

DGVPrinter printer = new DVGPrinter ();
print.OwnerDraw += new CellOwnerDrawEventHandler(print_OwnerDraw);
print.RowHeight = DGVPrinter.RowHeightSetting.CellHeight;

...

// Draw column headers upside down and backwards
public void OwnerDraw(object sender, DGVCellDrawingEventArgs e)
{
    if (e.row == -1)
    {
        DataGridViewHeaderCell cell = RequestList.Columns[e.column].HeaderCell;
        String printvalue = RequestList.Columns[e.column].HeaderText;

        // allow the user to do whatever

        // draw background
        e.g.FillRectangle(new SolidBrush(e.CellStyle.BackColor), e.DrawingBounds);

        // Draw column header text upside down and backwards
        e.g.TranslateTransform(e.DrawingBounds.X + e.DrawingBounds.Width,
            e.DrawingBounds.Y + e.DrawingBounds.Height);
        e.g.RotateTransform(180);
        e.g.DrawString(printvalue, e.CellStyle.Font,
            new SolidBrush(e.CellStyle.ForeColor), e.CellStyle.Padding.Left, -
cell.InheritedStyle.Padding.Bottom);

        // undo the backwards upside down transform
        e.g.ResetTransform();

        // draw grid
        if (RequestList.CellBorderStyle != DataGridViewCellBorderStyle.None)
            e.g.DrawRectangle(new Pen(RequestList.GridColor), e.DrawingBounds.X,
e.DrawingBounds.Y,
            e.DrawingBounds.Width, e.DrawingBounds.Height);

        e.Handled = true;
    }
}

```

Figure 13. Creating and using an owner-draw delegate